OS Orange – 2
Sanjith Suresh Kumar
PES1UG24AM250

1A :

```
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B :

```
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ find .pes/objects -type f
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/15/65fad0fc313fe0e4d263a68f689a8362490c600727d356af05e9c97041c051
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/d6/84632d35b8435a478924072c53daeab8944ddd3023cd6ffc8c8656d5615275
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/0c/fecdb3770772a9fed1684147880d6f8b4ead0eaf56a112e8af95a32608438f
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$
```

2A :

```
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B :

```
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ xxd .pes/objects/96/c0d5c28dd3b719
606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
00000000: 7472 6565 2031 3934 0031 3030 3634 3420  tree 194.100644
00000010: 6279 652e 7478 7400 e67e d66b cb4a 7082  bye.txt..~.k.Jp.
00000020: 50a8 9f31 5d4d 8bb9 2703 343c 84df 8344  P..1]M..'.4<...D
00000030: 3888 0e13 a686 52bc 3130 3036 3434 2066  8.....R.100644 f
00000040: 696c 6531 2e74 7874 002c f8d8 3d9e e295  ile1.txt.,..=...
00000050: 43b3 4a87 7274 21fd ecb7 e3f3 a183 d337  C.J.rt!........7
00000060: 6390 25de 576d b9eb b431 3030 3634 3420  c.%.Wm...100644
00000070: 6669 6c65 322e 7478 7400 e00c 50e1 6a2d  file2.txt...P.j-
00000080: f38f 8d6b f809 e181 ad02 48da 6e67 19f3  ...k......H.ng..
00000090: 5f9f 7e65 d6f6 0619 9f7f 3130 3036 3434  _.~e......100644
000000a0: 2068 656c 6c6f 2e74 7874 0005 c8d7 77d0   hello.txt....w.
000000b0: be85 748b 6788 d1b5 caf7 d697 54e6 e680  ..t.g.......T...
000000c0: 60c2 65ca 21cb d6c0 5e5f 43              `.e.!...^_C
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$
```

3A :

```
sanjith@sanjith-Yoga-Pro-9-16IMH9: ~/Orange_4/PES1UG24AM250-pes-vcs
All Phase 2 tests passed.
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ make pes
make: 'pes' is up to date.
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ echo "hello" > file1.txt
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ echo "world" > file2.txt
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ ./pes add file1.txt file2.txt
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_objects.c
  untracked:  test_tree
  untracked:  pes.h
  untracked:  commit.c
  untracked:  test_objects
  untracked:  pes.c
  untracked:  index.c
  untracked:  commit.h
```

3B :

```
sanjith@sanjith-Yoga-Pro-9-16IMH9: ~/Orange_4/PES1UG24AM250-pes-vcs
  untracked:  pes.h
  untracked:  commit.c
  untracked:  test_objects
  untracked:  pes.c
  untracked:  index.c
  untracked:  commit.h
  untracked:  .gitignore
  untracked:  object.c
  untracked:  test_sequence.sh
  untracked:  tree.c
  untracked:  tree.h
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  README.md
  untracked:  Makefile

sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776334320 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776334327 6 file2.txt
```

4A :

```
sanjith@sanjith-Yoga-Pro-9-16IMH9: ~/Orange_4/PES1UG24AM250-pes-vcs
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: 0cfecdb37707... Initial commit
Committed: 1565fad0fc31... Add world
Committed: d684632d35b8... Add farewell
commit d684632d35b8435a478924072c53daeab8944ddd3023cd6ffc8c8656d5615275
Author: PES User <pes@localhost>
Date:   1776334760

    Add farewell

commit 1565fad0fc313fe0e4d263a68f689a8362490c600727d356af05e9c97041c051
Author: PES User <pes@localhost>
Date:   1776334760

    Add worldell

commit 0cfecdb3770772a9fed1684147880d6f8b4ead0eaf56a112e8af95a32608438f
Author: PES User <pes@localhost>
Date:   1776334760

    Initial commit
```

4B :

```
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/0c/fecdb3770772a9fed1684147880d6f8b4ead0eaf56a112e8af95a32608438f
.pes/objects/15/65fad0fc313fe0e4d263a68f689a8362490c600727d356af05e9c97041c051
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/d6/84632d35b8435a478924072c53daeab8944ddd3023cd6ffc8c8656d5615275
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
.pes/refs/heads/main
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$
```

4C :

```
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ cat .pes/refs/heads/main
d684632d35b8435a478924072c53daeab8944ddd3023cd6ffc8c8656d5615275
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
sanjith@sanjith-Yoga-Pro-9-16IMH9:~/Orange_4/PES1UG24AM250-pes-vcs$
```

Questions answers –
Q5.1: How would you implement pes checkout <branch>?
A)To implement pes checkout <branch>, three things must change inside .pes/:

1. Read the commit hash from .pes/refs/heads/<branch>
2. Update .pes/HEAD to contain ref: refs/heads/<branch>
3. Rewrite the index file to reflect the target commit's tree

For the working directory, the system must walk the target commit's root tree, compare each file against the current working directory, then create, overwrite, or delete files as needed so the working directory matches the target snapshot.
The operation is complex because it must be atomic across many files — a crash mid-checkout can leave a mix of old and new files, the index can go out of sync with the working directory, untracked local files must not be destroyed, and conflicts with uncommitted changes must be detected before touching anything
Q5.2 — Detecting a dirty working directory
A) A file is "dirty" if it differs from both the index and the target branch's tree. To detect this, iterate through each entry in the current index. For each path, compare three versions:

1. **Index hash** — from .pes/index
2. **Working directory hash** — recomputed by reading the actual file and hashing it (or skip the hash and compare mtime + size for a fast check)
3. **Target tree hash** — from the tree of the branch being checked out to

If working directory hash ≠ index hash (file modified locally but not staged) AND index hash ≠ target tree hash (file also differs between branches), checkout must refuse because applying the target would silently discard the local changes.
Q5.3 — Detached HEAD

A) Detached HEAD means .pes/HEAD contains a raw commit hash instead of ref: refs/heads/<branch>. Commits made in this state still write normally to the object store and head_update still overwrites HEAD with the new commit hash — but since no branch ref points to these commits, they are "floating." If the user switches to another branch, those commits become unreachable (no branch sees them) and will be deleted on the next garbage collection.

Recovery: before switching branches, the user must note the commit hash (via pes log or the current value of HEAD) and create a new branch pointing to it — e.g., pes branch rescue <hash> — which writes the hash into .pes/refs/heads/rescue, making those commits reachable again.
Q6.1 — Garbage collection algorithm

A) Algorithm: **mark and sweep**.

1. **Mark phase:** start from all branch refs (.pes/refs/heads/*) and HEAD. For each ref, read its commit, mark the commit hash, then recursively mark its tree, all subtrees inside the tree, all blobs referenced by those trees, and finally follow the commit's parent pointer. Continue until every reachable object is marked.
2. **Sweep phase:** walk .pes/objects/ on disk. For every object file, check if its hash is in the "reachable" set. If not, delete it.

The ideal data structure for the reachable set is a **hash set (open-addressed hash table keyed by the 32-byte ObjectID)**, giving O(1) lookup.
For a repo with 100,000 commits and 50 branches: each commit references ~1 tree and on average a few subtrees/blobs. Assuming branches share most history and an average of ~5 objects referenced per commit, the walk visits roughly 100,000 commits + 500,000 trees/blobs ≈ **600,000 objects**. The 50 branches add little extra work because they share ancestry — the walk simply starts from 50 tips instead of 1.

Q6.2 — GC vs. concurrent commit race condition

A) Race condition: a commit operation has three steps — write blob objects, write a tree, write a commit. There is a window between writing a blob (say hello.txt) and writing the tree that references it. If GC runs during that window, it sees the blob on disk but finds no tree or commit referring to it, marks it as unreachable, and deletes it. The commit then writes a tree pointing to a blob hash that no longer exists, producing a corrupt repository.

Real Git avoids this using three mechanisms:

1. **Reachability grace period** — objects are only considered for deletion if their mtime is older than a cutoff (default two weeks), so freshly written blobs are never collected.
2. **Locks** — GC acquires a repository-wide lock (gc.pid), and commit operations check for it before writing.
3. **Reflog** — every ref update is logged, so even objects not yet referenced by a commit are kept reachable via the reflog entry until the reflog expires.